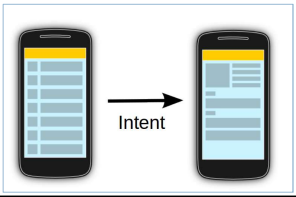
146

Chapitre 4

Intents



Naoufel KHAYATI
naoufel.khayati@eniso.u-sousse.tn

AU. 2025 - 2026

Introduction

147

- La communication au sein d'un système Android est basée sur l'échange de messages.
  - Un message = Une intention d'une action.
- Un message peut être émis à un autre composant de la même application (une activité, un service, etc.) ou d'une autre application.
- Un message permet de véhiculer toutes les informations nécessaires à la réalisation d'une action:
  - action à effectuer et données à utiliser.

→ **Les Intents**

Introduction

148

- Les objets ***Intent*** servent essentiellement à:
  - Démarrer une activité au sein de la même application
  - Solliciter d'autres applications
  - Diffuser des informations
- Pour le 1er point, on parle de démarrage d'une activité en *mode explicite*.
- Pour le 2ème point, on parle de démarrage d'une activité en *mode implicite* et de *résolution d'intents*.
- Pour le 3ème point, on parle de *diffusion d'intents* (non traitée dans ce cours).

149

Intents explicites

Démarrage d'une activité – Sans retour

150

- De façon générale, chaque composant d'une application nécessite l'emploi d'un Intent pour être démarré.
- Si on souhaite démarrer une activité sans retour, on utilise simplement la méthode **startActivity()**
  - Son paramètre est une instance de la classe ***Intent*** spécifiant la classe de l'activité à lancer :

```
val i = Intent(v?.context,CalledActivity::class.java)
startActivity(i)
```

Démarrage d'une activité – Avec retour

151

- Notons « activité fille », l'activité à lancer.
- Notons « activité mère », celle qui va démarrer l'activité fille.

- Avec **startActivity()**, on ne sera jamais averti de l'état de l'activité fille.
  - On ne peut rien récupérer de cette activité.
- Si par exemple une activité fille propose à l'utilisateur de répondre à un formulaire par *Oui ou Non*.
  - Comment l'activité mère récupère-t-elle la valeur saisie dans l'activité fille?

Démarrage d'une activité – Avec retour

152

- Afin de permettre à une activité mère de récupérer un résultat de la part d'une activité fille, il faut suivre les étapes suivantes :
  - enregistrer l'activité mère pour la récupération d'un résultat,
  - lancer l'activité fille qui fournira un résultat, et
  - traiter le résultat une fois reçu par l'activité mère.

Démarrage d'une activité – Avec retour

153

1. Enregistrement de l'activité mère pour la récupération d'un résultat

```
var theLauncher = registerForActivityResult(ActivityResultContracts.StartActivityForResult()) { it: ActivityResult!
    // Handle the received result
}
```

- ***registerForActivityResult()*** renvoie un ***ActivityResultLauncher*** qui sera utilisé pour lancer l'activité fille.
  - Dedans, c'est le code de la fonction de callback ***onActivityResult()*** qui servira à traiter le résultat reçu.

Démarrage d'une activité – Avec retour

154

- Si on attend plusieurs résultats distincts, on peut appeler ***registerForActivityResult()*** plusieurs fois pour enregistrer plusieurs instances ***ActivityResultLauncher***.
- ***registerForActivityResult()*** peut être appelée en toute sécurité avant la création de l'activité (en dehors de ***onCreate()***).

Démarrage d'une activité – Avec retour

155

2. Lancement de l'activité fille
   - Se fait grâce à la méthode launch() de l'objet ***ActivityResultLauncher*** précédemment créé.

```
intent= Intent(applicationContext, CalledActivity::class.java)
theLauncher.launch(intent)
```

3. Traitement du résultat
   - Lorsque l'activité fille aura terminé sa tâche, elle en avertira l'activité mère en lui communiquant un code résultat (par exemples **RESULT_OK** ou **RESULT_CANCELED**) et éventuellement un résultat.
   - Ensuite, la fonction de callback ***onActivityResult()*** sera exécutée.

Démarrage d'une activité – Avec retour

156

- Un **resultCode** peut prendre plusieurs valeurs:
  - *RESULT_OK* vaut -1 : si l'activité fille va jusqu'au bout de son exécution.
  - *RESULT_CANCELED* vaut 0 : si l'activité fille a été interrompue (bouton Cancel / Annuler ou appui sur la touche retour arrière du téléphone).
  - RESULT_FIRST_USER : pour commencer une liste de valeurs définies par l'utilisateur.

Démarrage d'une activité – Avec retour

157

- Dans l'activité fille, c'est la méthode **setResult()** qui renverra le **resultCode** et le résultat vers l'activité mère.
- **setResult()** est toujours suivie par la méthode **finish()**, pour quitter l'activité fille.

```
binding.btn.setOnClickListener { it: View!
    setResult(RESULT_OK)
    finish()
}
```

Transport d'informations – (M → F)

158 Mère

Transmission

- Les intents peuvent permettre aussi de "transporter" des informations grâce à la méthode **putExtra ()**.
  - Cette méthode utilise le principe d'un **couple clé/valeur.**
- La méthode "**putExtra ()**" est appliquée sur un objet de type ***Intent***.
  - L'intent va donc appeler l'activité fille et lui transmettre l'information.

intent.putExtra (clé,valeur)

Transport d'informations – (M → F)

159 Mère

Transmission – Code chez la mère

```
binding.button.setOnClickListener {  it: View!
        intent= Intent(applicationContext, CalledActivity::class.java)
        intent.putExtra( name: "nom", binding.editName.toString())
        intent.putExtra( name: "nombre",  value: 20)
        theLauncher.launch(intent)
    }
```

Transport d'informations – (M → F)

160 Fille

Extraction

- Extraire la valeur de l'intent: il faut utiliser un objet de type "Bundle".
  - val extras = intent.extras
- Placer la valeur transportée dans une variable: il faut appliquer la méthode "get*X*" à l'objet "Bundle".
  - Cette méthode accepte un paramètre, la clé associée à la valeur transportée.
  - *X* est à remplacer par le type de la valeur transportée.
    - "getString" si la valeur transportée est de type String,
    - "getInt" si la valeur est de type int,
    - "getDoubleArray" si la valeur transportée est un tableau de type Double,
    - etc.

Transport d'informations – (M → F)

161 Fille

Extraction – Code chez la fille

NB : t1 et t2 sont deux TextViews

```
val extras = intent.extras
val nom = extras?.getString("nom")
val nombre = extras?.getInt("nombre"
t1.text = nom
t2.text = "$nombre"
```

```
val nom = intent.getStringExtra("nom")
val nombre = intent.getIntExtra("nombre",0)
```

Transport d'informations – (F → M)

162

- Créer un objet de type **Intent** vide dans l'activité fille.
- Utiliser la méthode **putExtra()** pour remplir l'objet Intent avec les informations à transmettre.
- La méthode **setResult()** aura alors 2 paramètres
  - Le **resultCode** et,
  - l'objet **Intent** déjà créé.
- L'activité mère utilisera la méthode **get*X*Extra()**.

Transport d'informations – (F → M)

163 Fille

Code chez la fille

```
binding.btn.setOnClickListener {  it: View!
    var i= Intent()
    i.putExtra( name: "res",  value: "Voici le résultat")
    setResult(RESULT_OK, i)
    finish()
}
```

Transport d'informations – (F → M)

164 Mère

Code chez la mère

```
var theLauncher = registerForActivityResult(ActivityResultContracts.StartActivityForResult()) {
    when (it.resultCode) {
        RESULT_OK -> binding.txtres.text = it.data?.getStringExtra( name: "res")
        RESULT_CANCELED -> Toast.makeText(
            applicationContext,
            text: "Action annulée !!!",
            Toast.LENGTH_LONG
        ).show()
    }
}
```

165

Intents implicites

Appel à une autre application

166

- Une application A, pour solliciter une autre application B (inconnue pour A):
  - peut se contenter de transmettre son intention au système qui va se charger de:
    - trouver l'application B et le composant correspondant,
    - démarrer ce dernier (activité, service, etc.),
    - lui transmettre l'Intent correspondant.

Appel à une autre application

167

- C'est le système qui décide d'utiliser l'application adéquate.
  - Navigateur pour ouvrir une page web
  - Composeur pour passer un appel
- Pour choisir le composant le plus approprié, le système se base sur les informations spécifiées dans:
  - l'objet Intent (action, données, etc.) créé par le demandeur.
  - les Intent Filters définis dans le fichier Manifest du composant recherché.
    - Il essaie de trouver une correspondance.
- Ce mécanisme d'**Intent implicite**, permet d'éviter les dépendances envers des applications.
  - L'association entre l'application appelante et le composant appelé se fait au moment de l'exécution et non pas à la compilation.

Appel à une autre application

168

Les données

- Les données sont formatées à l'aide des URI.
  - Un URI est une chaîne de caractères qui permet d'identifier un endroit.
  - La syntaxe d'un URI se présente comme suit:

```
<schéma> : <information> { ? <requête> } { # <fragment> }
```

Appel à une autre application

169

```
<schéma> : <information> { ? <requête> } { # <fragment> }
```

- Le schéma décrit la nature de l'information.
  - **tel**: s'il s'agit d'un numéro de téléphone à composer.
  - **sms** / **smsto**: s'il s'agit d'un numéro de téléphone destinataire avec éventuellement le corps du message.
  - **http**: s'il s'agit d'un site internet.
  - **geo**: s'il s'agit d'une requête GPS (composée de latitude et longitude).
- L'information est la donnée elle même.
  - tel: **95200444**
  - geo: **36.8, 10.17**
- La requête permet de fournir une précision par rapport à l'information.
- Le fragment permet d'accéder à une sous-partie de l'information.

Appel à une autre application

170

L'action

- Une action est une constante qui se trouve dans la classe Intent et qui commence toujours par « ACTION_ ».
  - ACTION_VIEW: permet de visualiser une donnée.
    - Une adresse mail sera affichée dans l'application pour les mails,
    - Un numéro de téléphone dans le composeur,
    - Une adresse Web dans le navigateur, etc.

Appel à une autre application

171

- ACTION_DIAL: pour ouvrir le composeur de numéro du téléphone.
- ACTION_CALL: pour appeler un numéro de téléphone
- ACTION_SENDTO: pour envoyer un SMS à quelqu'un.
- ACTION_WEB_SEARCH: pour effectuer une recherche sur internet

- ACTION_DELETE: pour supprimer des données à partir d'un URI.
- ACTION_EDIT: pour ouvrir un éditeur adapté pour modifier les données fournies.
- ACTION_INSERT: pour insérer des données.
- ACTION_PICK: pour sélectionner un élément dans un ensemble de données.
  - Les 4 dernières actions fonctionnent avec des Content Providers.

Appel à une autre application

172

Exemple: Composer un numéro de téléphone.

- Créer un nouvel objet **Intent** spécifiant le type d'action et les valeurs complémentaires dans le constructeur;
  - ACTION_DIAL comme type d'action pour demander au système de composer un numéro.
  - Le numéro à appeler comme info complémentaire, inscrit dans un URI.
  - Aucune précision du type de la classe cible.

```
val appel = Intent(Intent.ACTION_DIAL, Uri.parse( uriString: "tel:20102030"))
```

Appel à une autre application

173

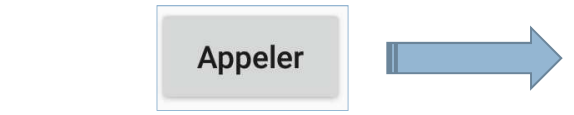
Exemple: Composer un numéro de téléphone.

- Démarrer une nouvelle activité en spécifiant cet objet Intent.
  - Le système déterminera quelle application ouvrir, par défaut l'interface de composition de numéro.

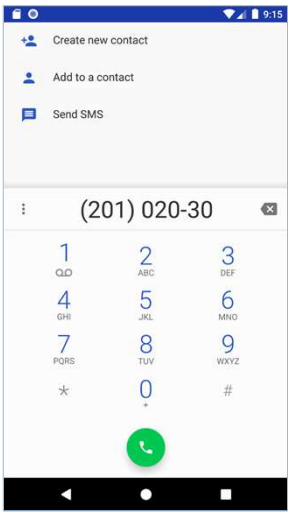
```
startActivity(appel)
```

Appel à une autre application

174

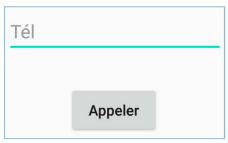



```
b.setOnClickListener { v: View? ->
    val uri = Uri.parse( uriString: "tel:20102030")
    val appel = Intent(Intent.ACTION_DIAL,uri)
    startActivity(appel)
}
```

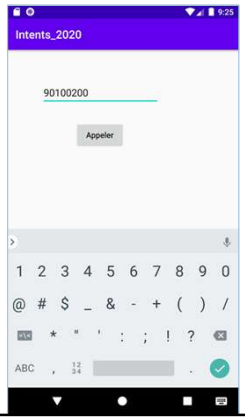


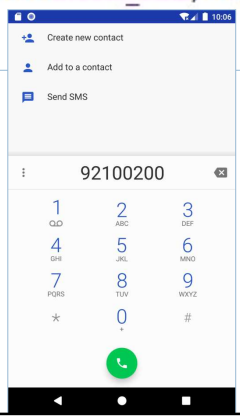
Appel à une autre application

175




```
val b1 = findViewById<Button>(R.id.button)
val e = findViewById<EditText>(R.id.editText2)
b1.setOnClickListener { v: View? ->
    val uri = Uri.parse( uriString: "tel:${e.text}")
    val appel = Intent(Intent.ACTION_DIAL,uri)
    startActivity(appel)
}
```



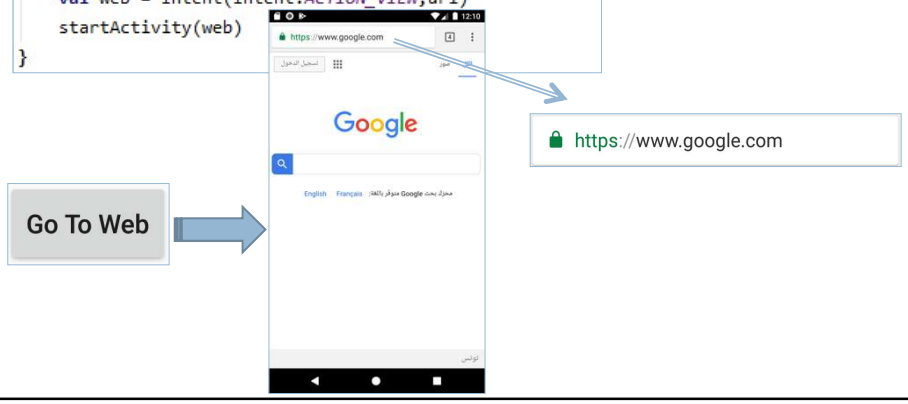


Appel à une autre application

176

- **Exemple: Visiter une page Web.**

```
val b1 = findViewById<Button>(R.id.button5)
b1.setOnClickListener { v: View? ->
    val uri = Uri.parse( uriString: "http://www.google.com")
    val web = Intent(Intent.ACTION_VIEW,uri)
    startActivity(web)
}
```



Appel à une autre application

177

Accorder les permissions liées aux actions

- émettre un appel depuis l'application

```
val b2 = findViewById<Button>(R.id.button6)
b2.setOnClickListener { it: View!
    val uri = Uri.parse( uriString: "tel:50101520")
    val appel = Intent(Intent.ACTION_CALL,uri)
    startActivity(appel)
}
```

Appel à une autre application

178

Accorder les permissions liées aux actions

- Certaines actions requièrent des privilèges spéciaux qui amèneront à ajouter des permissions dans le fichier ***Manifest*** de l'application.
- Pour notre exemple, il faudra ajouter la permission « *android.permission.CALL_PHONE* » sous l'élément racine <*manifest*>.

Appel à une autre application

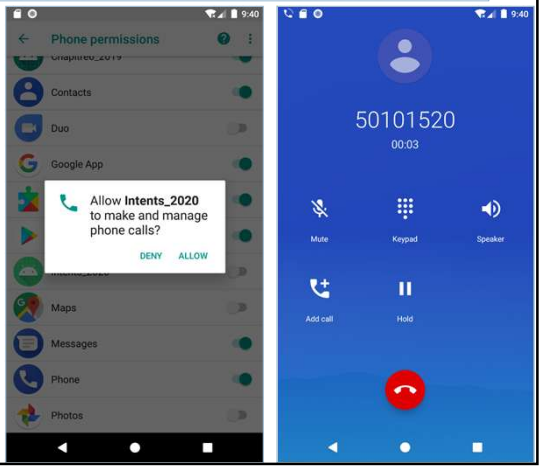
179

Accorder les permissions liées aux actions

```
<uses-permission android:name="android.permission.CALL_PHONE"/>
```

C'est insuffisant!!!
Il faut accorder l'autorisation manuellement dans l'appareil mobile, OU
Demander l'autorisation des utilisateurs.

Appel direct



Résolution d'Intents

180

- Par exemple, dans le cas de ACTION_VIEW, comment Android détermine quel composant doit répondre à un Intent donné?
- Android compare l'Intent à des filtres existants dans les **Manifest** des différentes applications installées.
  - Ces filtres signalent que les composants des applications peuvent gérer ou répondre à certains Intents.
  - Ces filtres sont représentés par les nœuds **<intent-filter>.**
- Un composant d'une application doit avoir autant de filtres que de capacités de traitement.
- Le test de conformité entre un Intent et un filtre peut se faire sur trois critères: **l'action, la catégorie** et **les données.**

Résolution d'Intents

181

- Exemples de Filtres (Manifest)

```
<activity
    android:name=".MainActivity"
    android:exported="true" >
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />

        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>
```

```
<activity
    android:name=".MyWebBrowserActivity"
    android:exported="true"
    android:theme="@style/Theme.App1102024" >
    <intent-filter>
        <action android:name="android.intent.action.VIEW" />

        <category android:name="android.intent.category.DEFAULT" />
        <category android:name="android.intent.category.BROWSABLE" />
        <category android:name="android.intent.category.APP_BROWSER" />

        <data android:scheme="http" />
        <data android:scheme="https" />
    </intent-filter>
</activity>
```

Résolution d'Intents

182

Action (filtre)

- Permet de filtrer en fonction du champ *Action* d'un Intent.
- Il peut y en avoir une ou plusieurs balises ***<action>*** par filtre.
  - Si un tel champ est absent, tous les Intents implicites seront refusés.
  - Un Intent sera accepté si ce qui se trouve dans son champ action est identique à au moins une des actions du filtre.
  - Si un Intent ne précise pas d'action, alors il sera automatiquement accepté.

Résolution d'Intents

183

- Exemples d'actions

```
<action android:name="android.intent.action.MAIN"/>

<action android:name="android.intent.action.VIEW"/>

<action android:name="android.intent.action.SEND"/>

<action android:name="android.intent.action.DIAL"/>

<action android:name="android.intent.action.CALL"/>

<action android:name="android.intent.action.WEB_SEARCH"/>

<action android:name="android.intent.action.EDIT"/>
```

Résolution d'Intents

184

Catégorie (filtre)

- Les balises **<*category*>** ne sont pas indispensables.
- S'il y en a une ou plusieurs, alors pour passer ce test, il faut que toutes les catégories de l'Intent correspondent à des catégories du filtre.
- Si on veut qu'un composant accepte les Intents implicites, on doit rajouter la catégorie CATEGORY_DEFAULT au filtre.

Résolution d'Intents

185

- Exemples de catégories

```
<category android:name="android.intent.category.DEFAULT"/>

<category android:name="android.intent.category.BROWSABLE"/>

<category android:name="android.intent.category.LAUNCHER"/>

<category android:name="android.intent.category.APP_EMAIL"/>
```

Résolution d'Intents

186

Données (filtre)

- Les balises ***<data>*** déclarent les différents types de données acceptés, en utilisant un ou plusieurs attributs.

```
<data android:scheme="http" />
<data android:scheme="https" />
<data android:scheme="https" android:host="www.example.com" />
<data android:scheme="https" android:host="www.example.com" android:path="/login" />
<data android:scheme="https" android:host="www.example.com" android:pathPrefix="/user" />
<data android:scheme="https" android:host="www.example.com" android:pathPattern="/profile/.*" />
<data android:mimeType="image/*"/>
<data android:scheme="file" android:mimeType="application/pdf" />
<data android:scheme="http" android:mimeType="application/pdf" />
```

Résolution d'Intents

187

Données (filtre)

- Les types MIME peuvent être (liste non exhaustive) :

```
<data android:mimeType="text/plain"/>
<data android:mimeType="text/html"/>
<data android:mimeType="image/jpeg"/>
<data android:mimeType="image/*"/>
<data android:mimeType="video/mpeg"/>
<data android:mimeType="audio/*"/>
<data android:mimeType="application/zip"/>
<data android:mimeType="application/msword"/>
<data android:mimeType="application/pdf"/>
```

Pour Finir…

188

- Les intents représentent l'un des principaux composants d'une application Android.
  - Intents explicites (les plus simples à définir et à gérer).
  - Intents implicites et résolution des intents via les filtres.
  - Diffusion d'intents (***non traitée dans ce cours***).

189

A SUIVRE…

TO BE CONTINUED…